COMP4442 Service and Cloud Computing
Lab 5

Weiyang MA

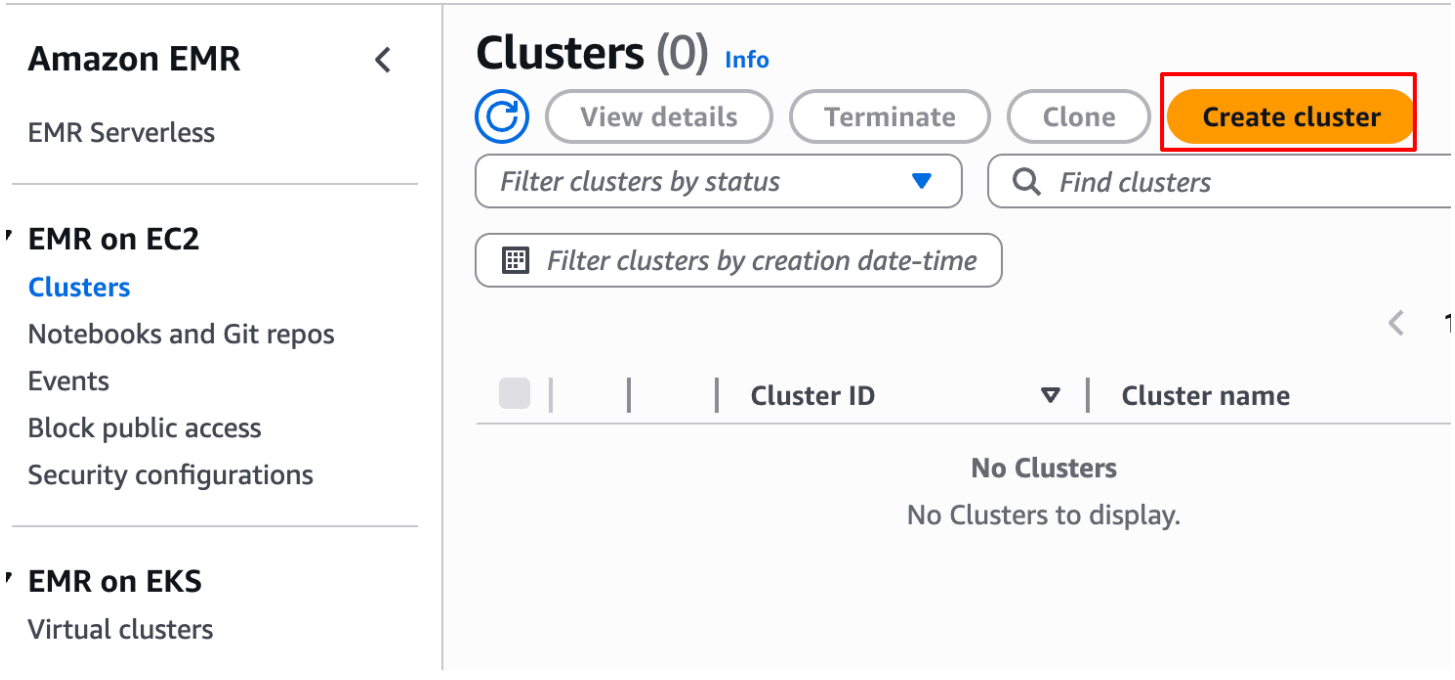
April 1, 2026

Outline

- Operations of Spark Resilient Distributed Datasets (RDD)
- How to use AWS Spark
  - Create Spark Cluster
  - Develop a spark program with python
  - Submit spark job to cluster

Create Spark Cluster

- Find the EMR service on the console, then create the spark cluster.



Create Spark Cluster

- Do the following configuration
  - Set cluster name
  - Choose **spark** at the cluster software
  - Choose **m4.large** instance type

Name

comp4442-lab5

Amazon EMR release Info

A release contains a set of applications which can be installed on your cluster.

emr-5.32.0

This EMR release reaches End of Support on Aug-01-2026 and will no longer be eligible for technical support. AWS strongly recommends that you run your workloads on the latest Amazon EMR release to receive security-critical updates and fixes. You can also use new Spark upgrade agent to modernize your existing applications on version 5.40 or higher to latest EMR version. To learn more, see EMR Standard Support policy and Spark Upgrades.

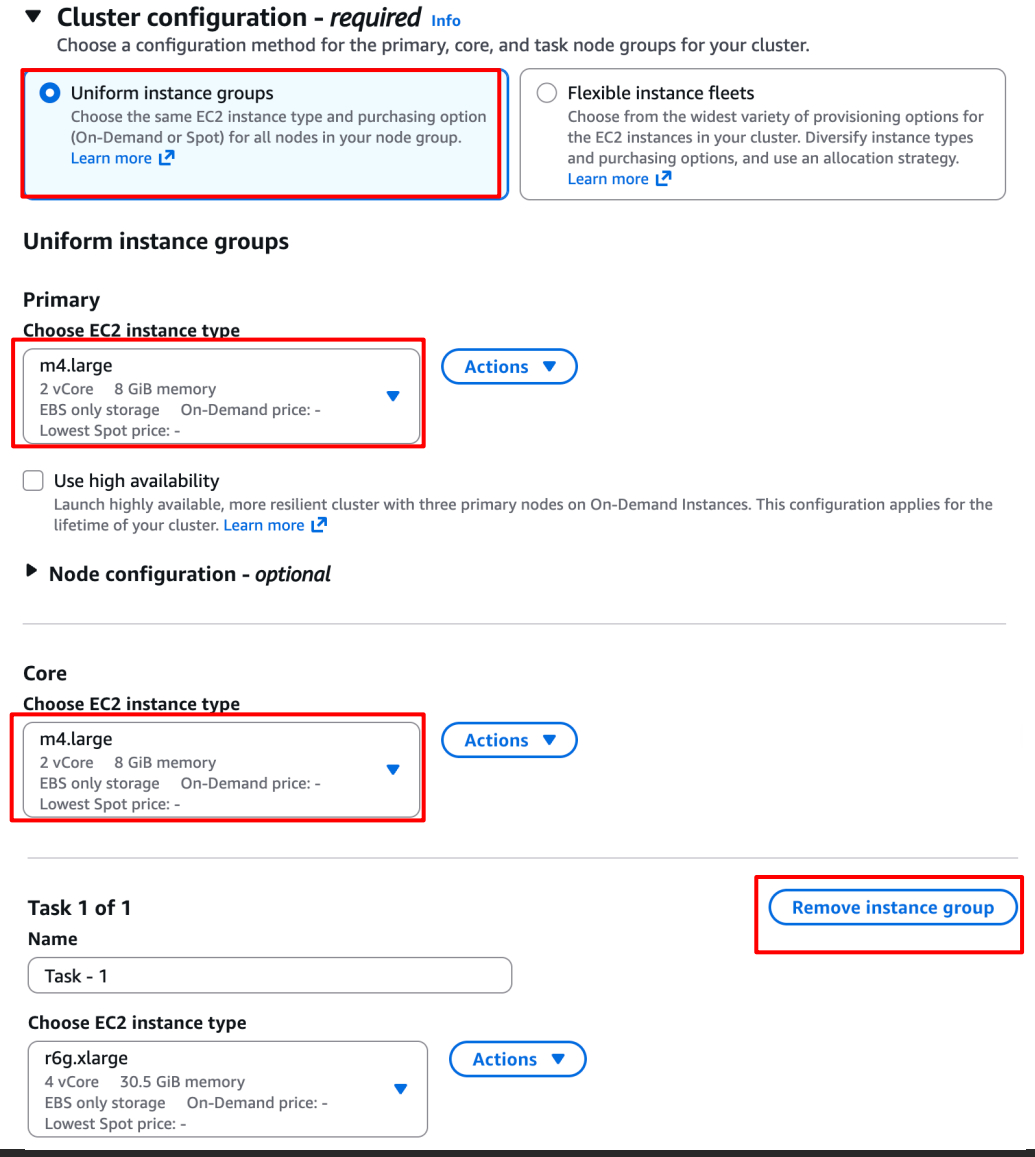
Application bundle

Spark | Core Hadoop | HBase | Presto | Custom

- [ ] Flink 1.11.2
- [ ] HCatalog 2.3.7
- [ ] Hue 4.8.0
- [ ] Livy 0.7.0
- [ ] Oozie 5.2.0
- [ ] Presto 0.240.1
- [ ] TensorFlow 2.3.1
- [ ] ZooKeeper 3.4.14
- [ ] Ganglia 3.7.2
- [ ] Hadoop 2.10.1
- [ ] JupyterEnterpriseGateway 2.1.0
- [ ] MXNet 1.7.0
- [ ] Phoenix 4.14.3
- [x] Spark 2.4.7
- [ ] Tez 0.9.2
- [ ] HBase 1.4.13
- [ ] Hive 2.3.7
- [ ] JupyterHub 1.1.0
- [ ] Mahout 0.13.0
- [ ] Pig 0.17.0
- [ ] Sqoop 1.4.7
- [x] Zeppelin 0.8.2

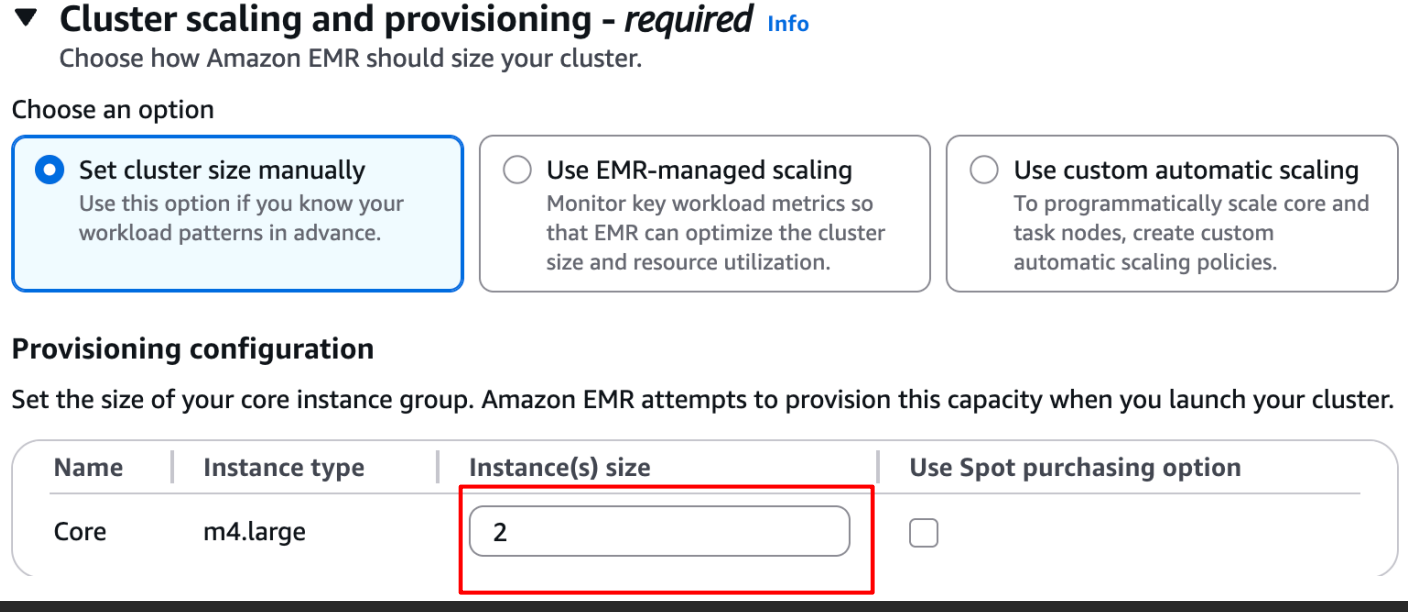
Create Spark Cluster

- Do the following configuration
  - Set cluster name
  - Choose **spark** at the cluster software
  - Choose **m4.large** instance type
  - Set a cluster with 1 primary node and 2 core nodes



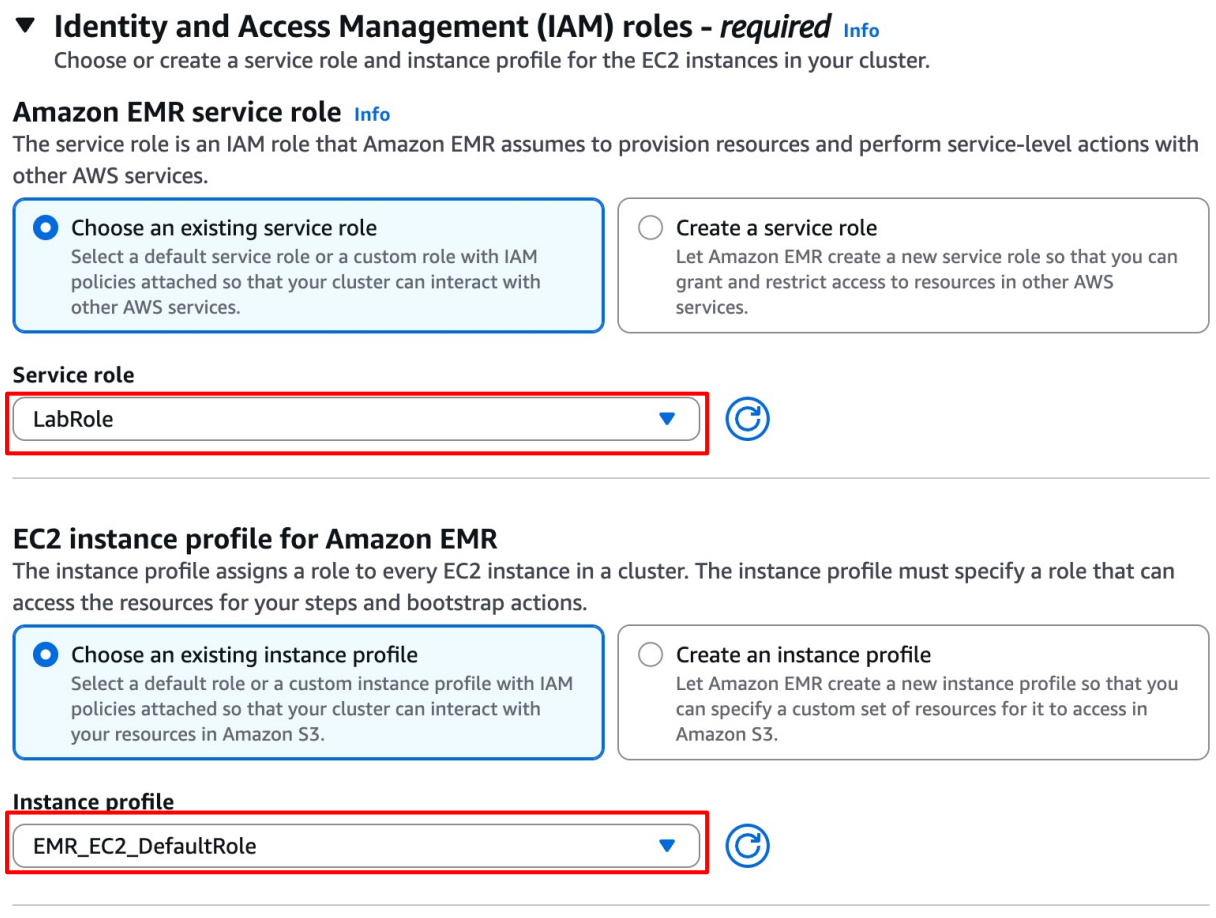
Create Spark Cluster

- Do the following configuration
  - Set cluster name
  - Choose **spark** at the cluster software
  - Choose **m4.large** instance type
  - Set a cluster with 1 primary node and 2 core nodes



Create Spark Cluster

- Do the following configuration
  - Set cluster name
  - Choose **spark** at the cluster software
  - Choose **m4.large** instance type
  - Leave other configurations as default



Operations of Spark RDD

- RDD is considered as the core of Spark
  - immutable and distributed collection of objects.
  - fundamental data structure, containing any type of *Python, Java, or Scala* objects, including user-defined classes.

- Spark program is essentially built on operations with RDD

- Two important operations on RDDs
  - Transformations
  - Actions.

Operations of Spark RDD

- Transformations are operations on the RDDs that create a new RDD by making changes to the original RDD.
  - **map()**: returns a new RDD by applying a function to each of the elements in the original RDD.
  - **flatMap()**: returns a new RDD by applying the function to every element of the parent RDD and then flattening the result.

```
data= [1, 2, 3]
myRDD= sc.parallelize(data)

mapRDD= myRDD.map(lambda x: range(1,x))
```

→ [[1], [1,2], [1,2,3]]

```
flatMapRDD = myRDD.flatMap(lambda x: range(1,x))
```

→ [1, 1, 2, 1, 2, 3]

Operations of Spark RDD

- Transformations are operations on the RDDs that create a new RDD by making changes to the original RDD.
  - **reduceByKey()**: called on a dataset of (key, value) pairs, returns a new dataset in which the values for each of its key are aggregated.
  - **collect()**: returns a list that contains all the elements of the RDD.(Actions)

```
from operator import add

myRDD = sc.parallelize([("a", 1), ("a", 2), ("a", 3), ("b", 1)])

#adds the values by keys
newRDD= myRDD.reduceByKey(add)
newRDD.collect()
```

→ [('a', 6), ('b', 1)]

Operations of Spark RDD

- Actions are the operations on the RDD to carry out computations and return the final result back to the driver.
  - **reduce()**: aggregates the elements of the RDD using a function that takes two elements of the RDD as input and gives the result.

```
data= ["Scala", "Python", "Java", "R"]
myRDD= sc.parallelize(data)

#Concatenate the string elements
myRDD.reduce( lambda x, y: x + y)
```

→ 'ScalaPythonJavaR'

Operations of Spark RDD

- Actions are the operations on the RDD to carry out computations and return the final result back to the driver.
  - **countByValue()**: returns the count of each unique value in the RDD as dictionary with (value, count) pairs.

```
data= ["Python", "Scala", "Python", "R", "Python", "Java", "R", ]

myRDD= sc.parallelize(data)

#items() returns a list with all the dictionary keys and values
returned by countByValue()

myRDD.countByValue().items()
```

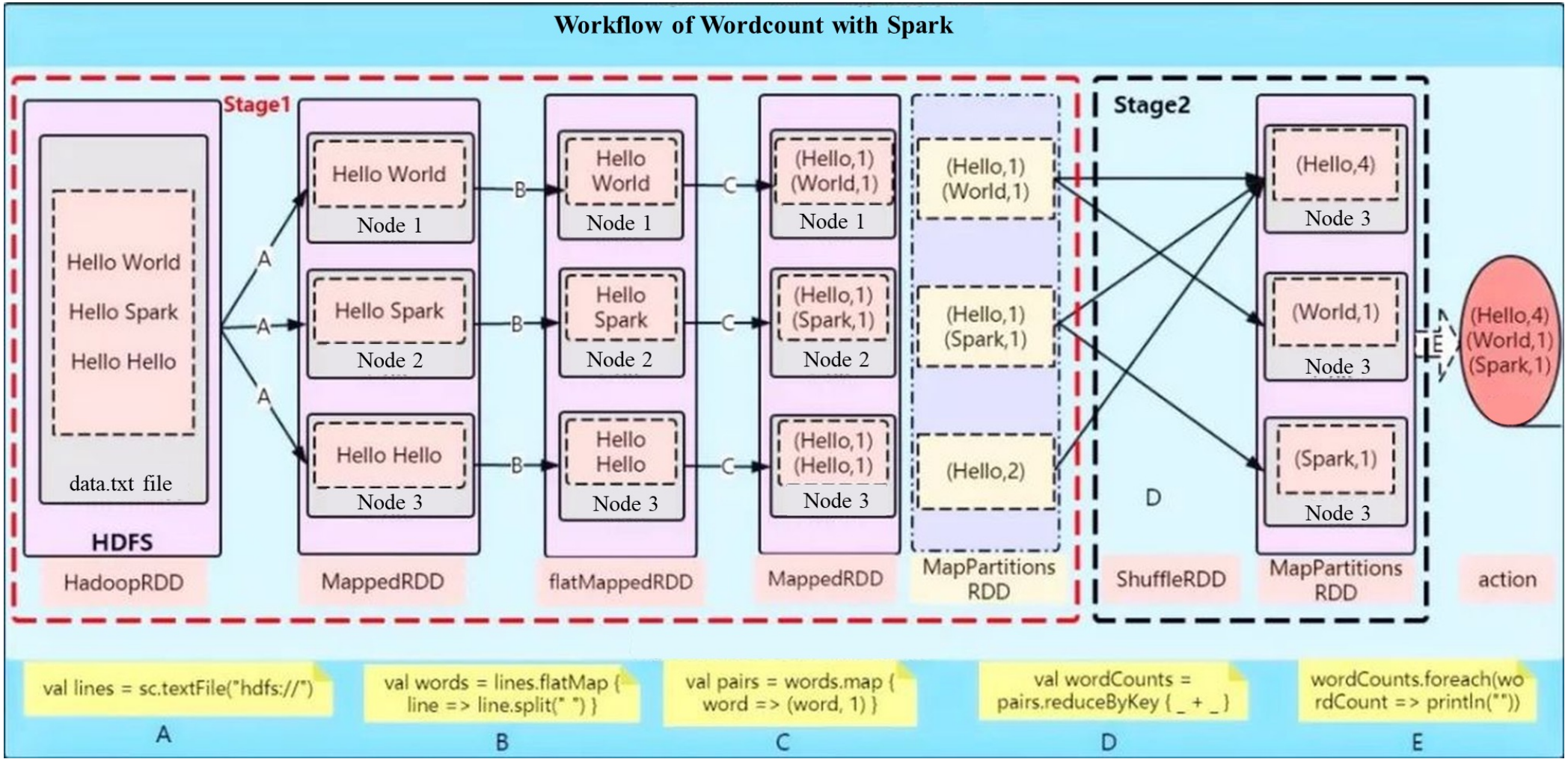
→ [('Python', 3), ('R', 2), ('Java', 1), ('Scala', 1)]

Operations of Spark RDD

Transformations	$map(f : T \Rightarrow U)$	:	$RDD[T] \Rightarrow RDD[U]$
	$filter(f : T \Rightarrow Bool)$	:	$RDD[T] \Rightarrow RDD[T]$
	$flatMap(f : T \Rightarrow Seq[U])$	:	$RDD[T] \Rightarrow RDD[U]$
	$sample(fraction : Float)$	:	$RDD[T] \Rightarrow RDD[T]$ (Deterministic sampling)
	$groupByKey()$	:	$RDD[(K, V)] \Rightarrow RDD[(K, Seq[V])]$
	$reduceByKey(f : (V, V) \Rightarrow V)$	:	$RDD[(K, V)] \Rightarrow RDD[(K, V)]$
	$union()$	:	$(RDD[T], RDD[T]) \Rightarrow RDD[T]$
	$join()$	:	$(RDD[(K, V)], RDD[(K, W)]) \Rightarrow RDD[(K, (V, W))]$
	$cogroup()$	:	$(RDD[(K, V)], RDD[(K, W)]) \Rightarrow RDD[(K, (Seq[V], Seq[W]))]$
	$crossProduct()$	:	$(RDD[T], RDD[U]) \Rightarrow RDD[(T, U)]$
	$mapValues(f : V \Rightarrow W)$	:	$RDD[(K, V)] \Rightarrow RDD[(K, W)]$ (Preserves partitioning)
	$sort(c : Comparator[K])$	:	$RDD[(K, V)] \Rightarrow RDD[(K, V)]$
	$partitionBy(p : Partitioner[K])$	:	$RDD[(K, V)] \Rightarrow RDD[(K, V)]$
Actions	$count()$	:	$RDD[T] \Rightarrow Long$
	$collect()$	:	$RDD[T] \Rightarrow Seq[T]$
	$reduce(f : (T, T) \Rightarrow T)$	:	$RDD[T] \Rightarrow T$
	$lookup(k : K)$	:	$RDD[(K, V)] \Rightarrow Seq[V]$ (On hash/range partitioned RDDs)
	$save(path : String)$	:	Outputs RDD to a storage system, *e.g.*, HDFS

https://medium.com/analytics-vidhya/understanding-spark-rdds-part-3-3b1b9331652a

Workflow of Wordcount with Spark



Develop Spark Program with Python

- Establish a *wordcount.py* file and upload to AWS S3

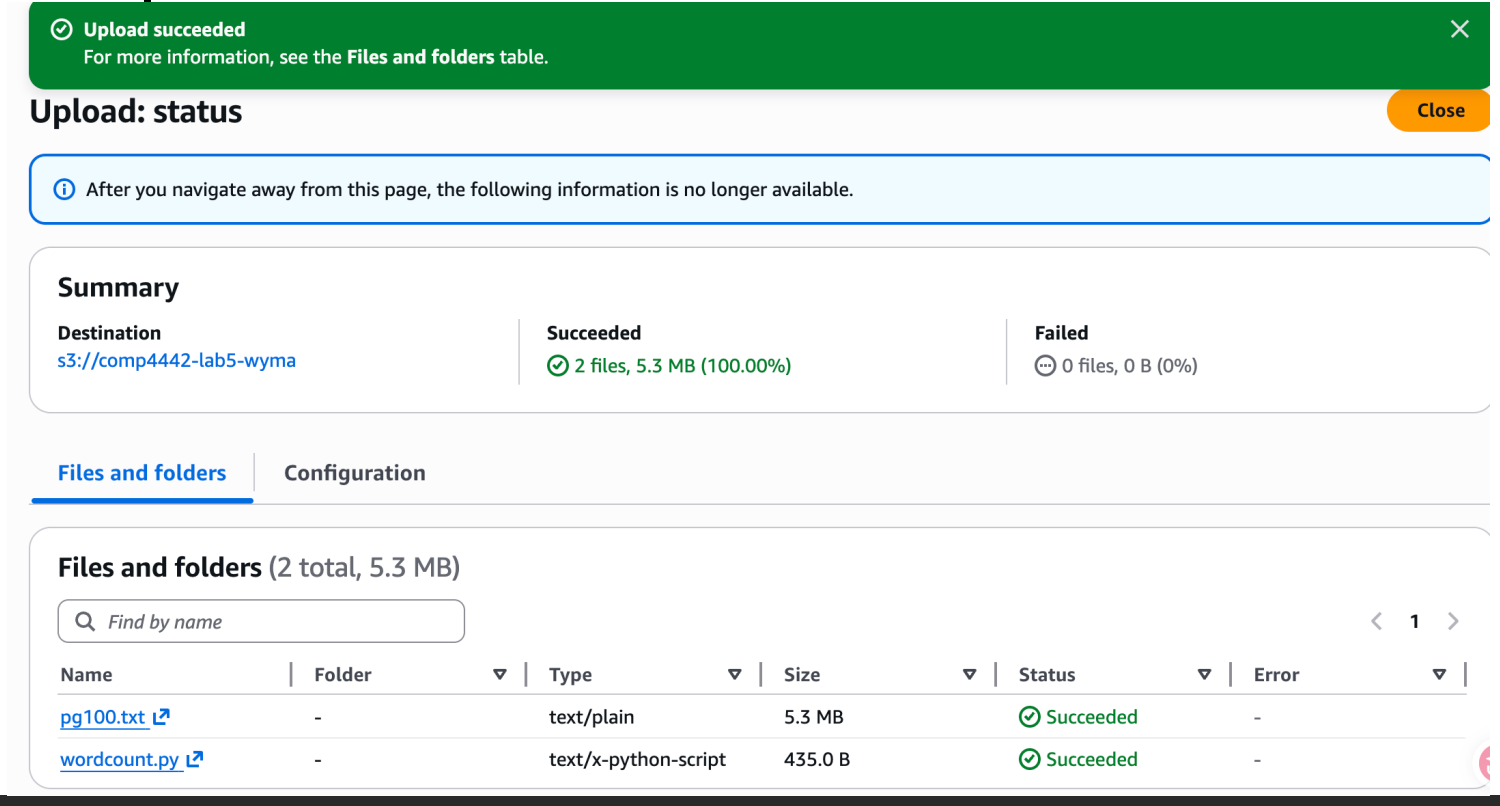
```
import os
import sys

from pyspark import SparkContext

args = sys.argv
inp = args[1]
out = args[2]

sc = SparkContext()

text_file = sc.textFile(inp)
counts = text_file.flatMap(lambda line: line.split(" ")) \
             .map(lambda word: (word, 1)) \
             .reduceByKey(lambda a, b: a + b)
counts.saveAsTextFile(out)

sc.stop()
```

Read system input parameters

Count frequency

Develop Spark Program with Python

- The *wordcount* example aims at counting the frequency of words in the input file
- Upload the input file to AWS S3



Submit Spark Job to Cluster

- On the spark cluster console, choose steps and then add step

Amazon EMR > EMR on EC2: Clusters > comp4442-lab5

comp4442-lab5 Updated less than a minute ago Terminate Clone in AWS CLI Clo

This EMR release reaches End of Support on Aug-01-2026 and will no longer be eligible for technical support. AWS strongly recommends that you run your workloads on the latest Amazon EMR release to receive security-critical updates and fixes. You can also use new Spark upgrade agent to modernize your existing applications on version 5.40 or higher to latest EMR version. To learn more, see EMR Standard Support policy and Spark Upgrades.

Summary

Cluster info

Cluster ID
j-9JVFZSX4TONS

Cluster ARN
arn:aws:elasticmapreduce:us-east-1:750349917420:cluster/j-9JVFZSX4TONS

Cluster configuration
Instance groups

Capacity
1 Primary | 2 Core | 0 Task

Applications

Amazon EMR version
emr-5.32.0

Installed applications
Spark 2.4.7, Zeppelin 0.8.2

Cluster management

Log destination in Amazon S3
aws-logs-750349917420-us-east-1/elasticmapreduce

Persistent application UIs
Spark History Server
YARN Timeline Server

Primary node public DNS
ec2-52-91-50-68.compute-1.amazonaws.com
Connect to the Primary node using SSH
Connect to the Primary node using SSM

Status and time

Status
Waiting

Creation time
March 31, 2026, 16:40 (U

Elapsed time
9 minutes, 5 seconds

Properties | Bootstrap actions | Instances (Hardware) | Steps | Applications | Configurations | Monitoring

Steps | Applications | Configurations | Monitoring

Refresh steps | leshoot with AI | Cancel | Clone | Add

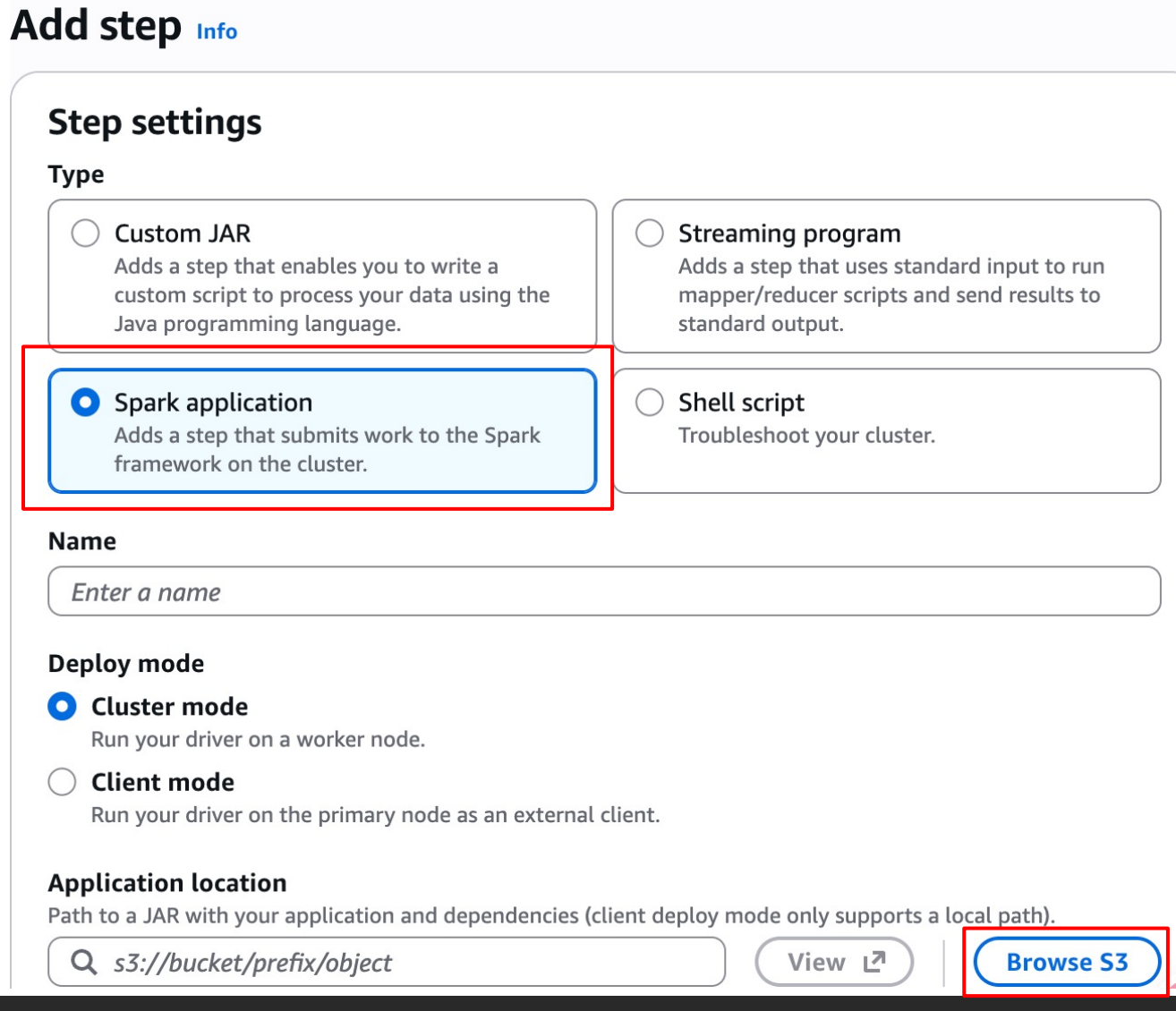
ssing by software installed on the cluster.

1

e | Log files | Creatio

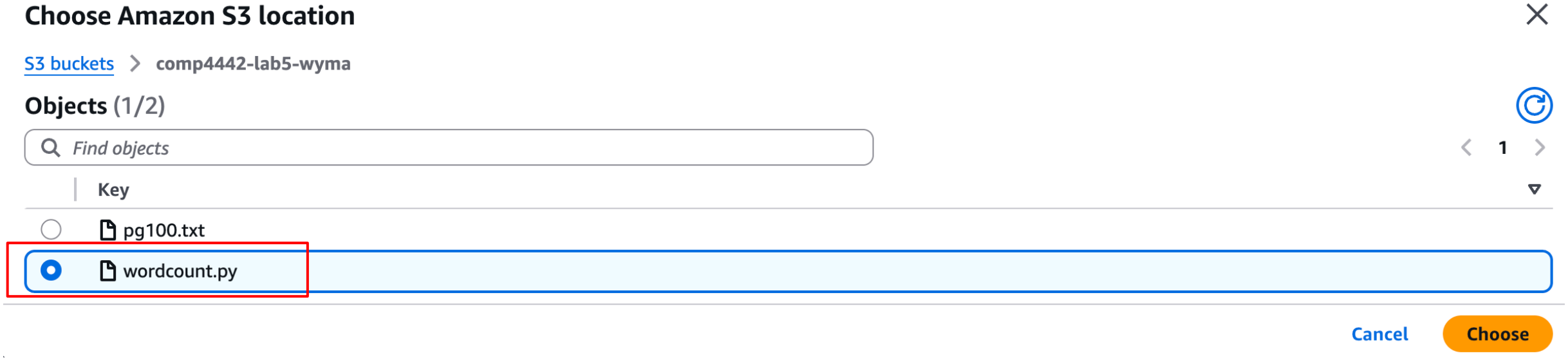
Submit Spark Job to Cluster

- Configure the spark job
  - Select Spark application as Step type
  - Indicate the application location
  - Set the application argument



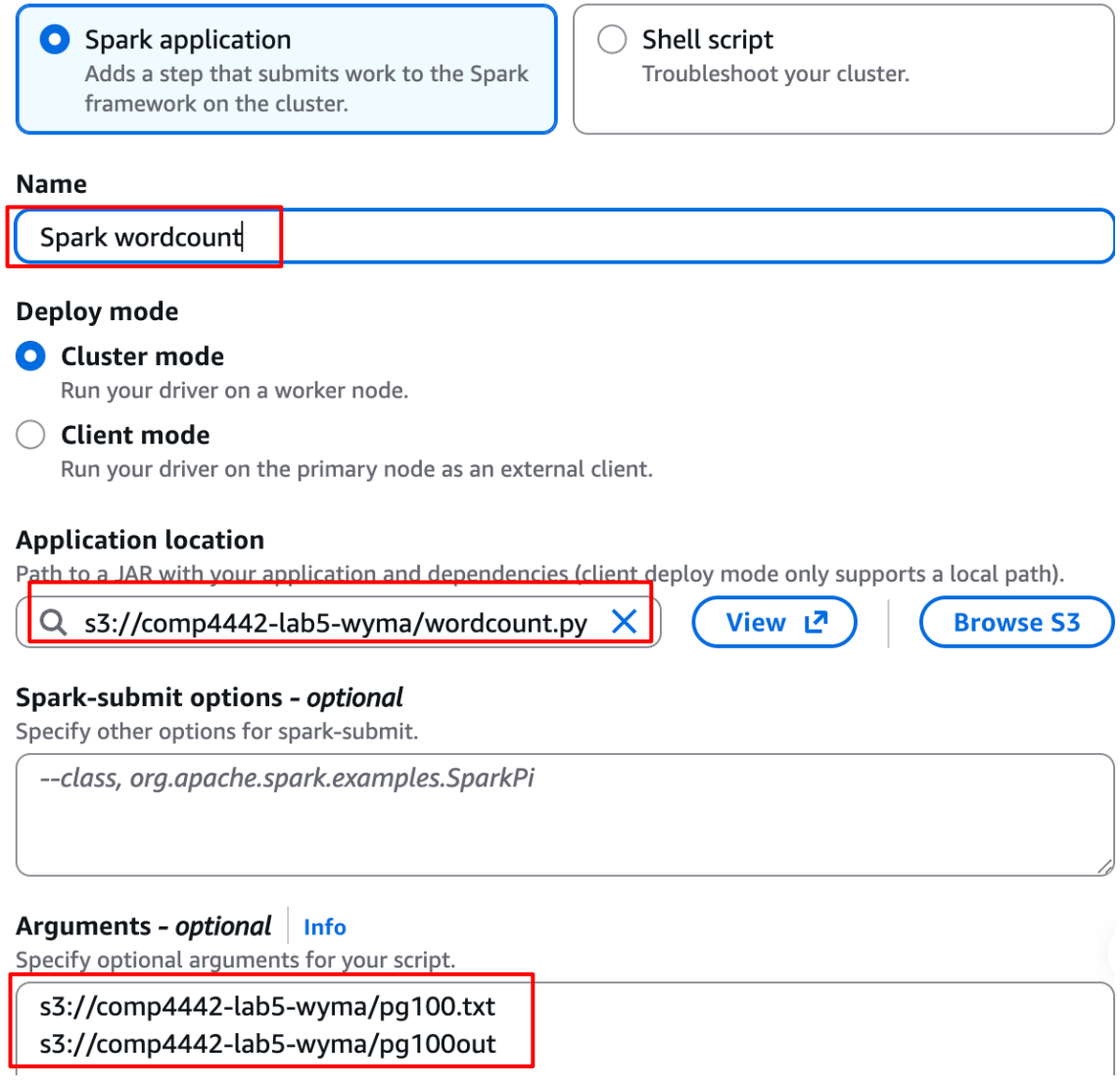
Submit Spark Job to Cluster

- Configure the spark job
  - Select Spark application as Step type
  - Indicate the application location
  - Set the application argument



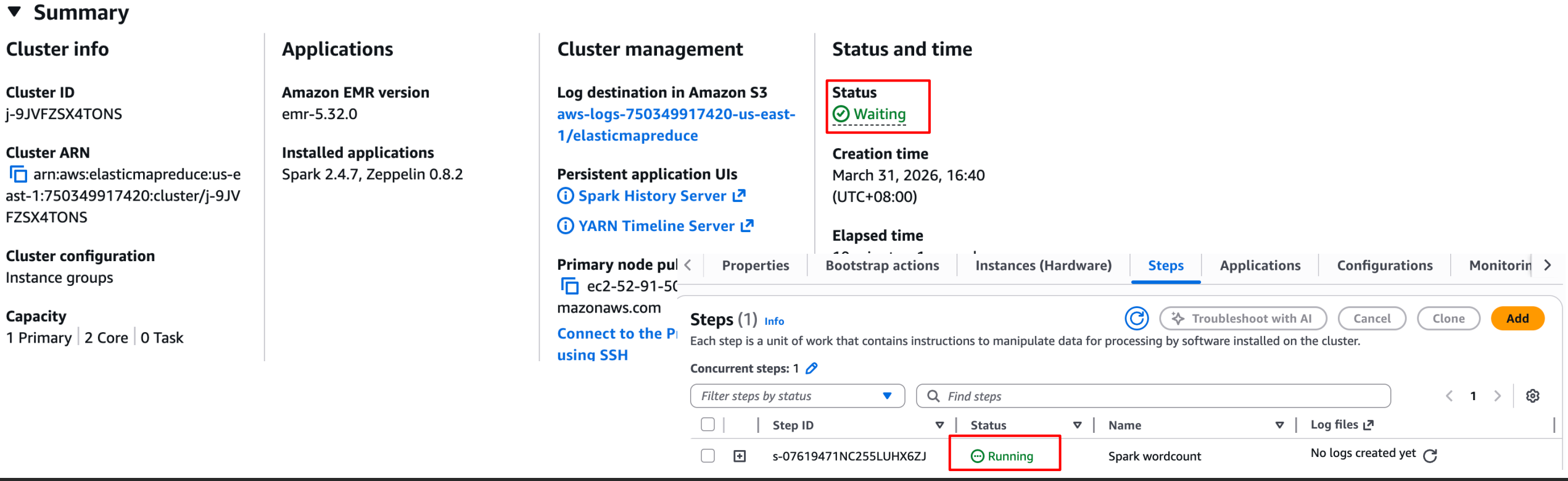
Submit Spark Job to Cluster

- Configure the spark job
  - Select Spark application as Step type
  - Indicate the application location
  - Set the application argument



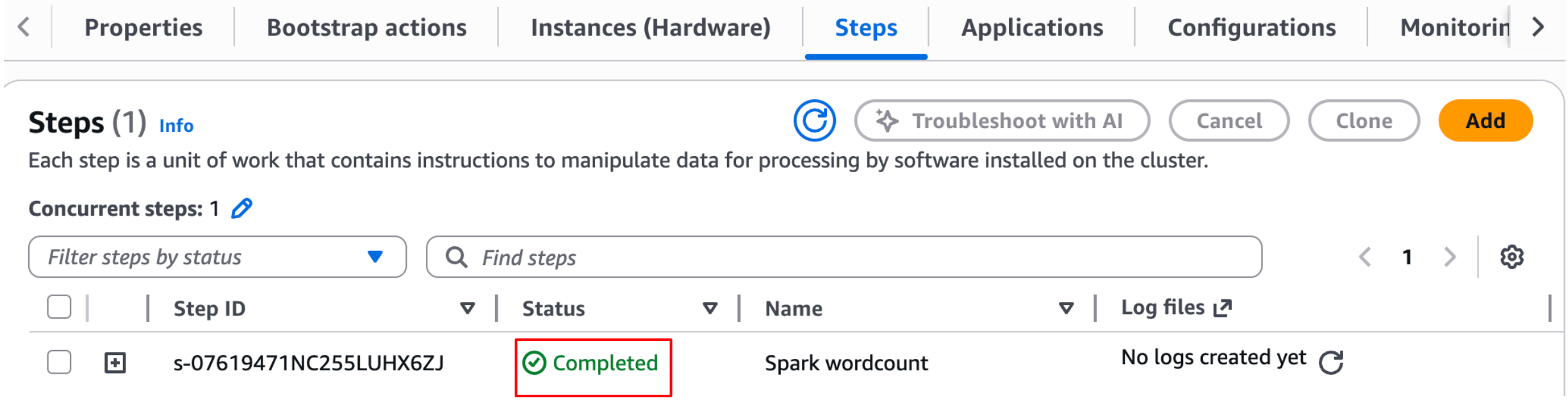
Submit Spark Job to Cluster

- Wait the cluster to run the job and save the output
  - Status of the cluster changes from starting to running, and then to wait (about 2 minutes)



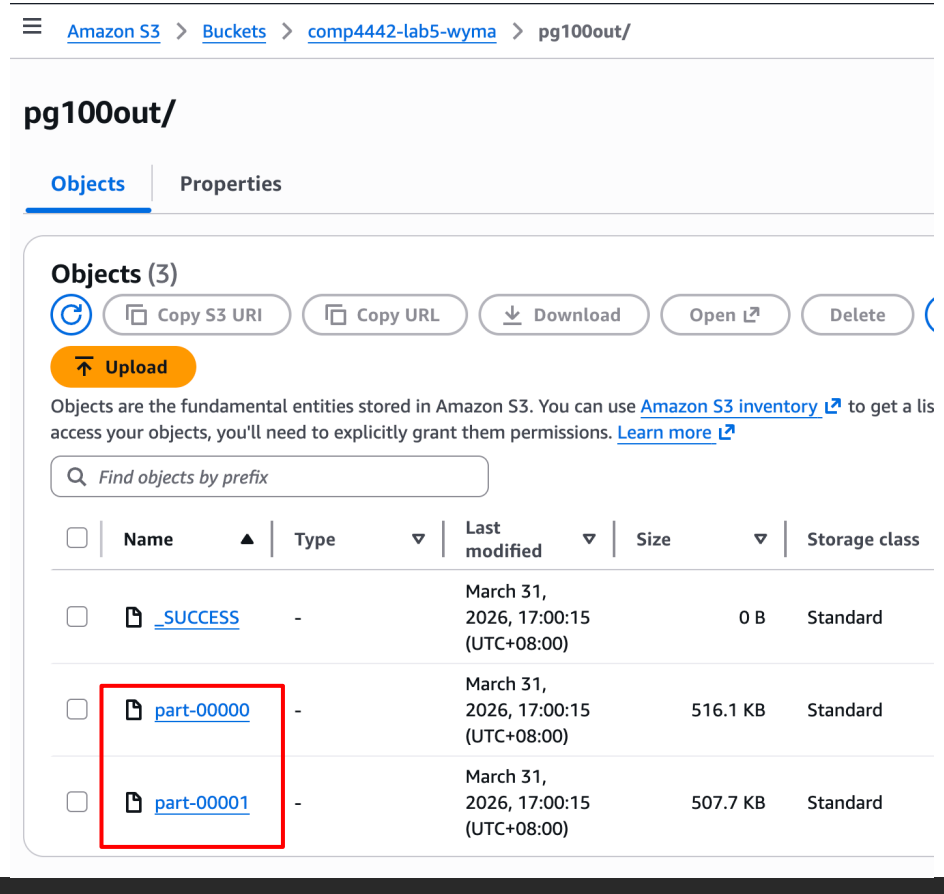
Submit Spark Job to Cluster

- Wait the cluster to run the job and save the output
  - Status of the cluster changes from starting to running, and then to wait (about 2 minutes)



Submit Spark Job to Cluster

- Check the output
  - Download output file from S3



part-00000 × part-00001

Users > kunstnaar > Downloads > part-00000

```
1  ('The', 3938)
2  ('Project', 9)
3  ('EBook', 1)
4  ('of', 15426)
5  ('Shakespeare', 43)
6  ('', 504352)
7  ('is', 7819)
8  ('use', 258)
9  ('anyone', 1)
10 ('anywhere', 3)
11 ('at', 2206)
12 ('no', 2423)
13 ('restrictions', 1)
14 ('whatsoever.', 2)
15 ('may', 1316)
16 ('it,', 524)
17 ('give', 960)
18 ('away', 293)
19 ('re-use', 1)
20 ('this', 4752)
21 ('online', 1)
22 ('www.gutenberg.org', 1)
```

Using command line tools

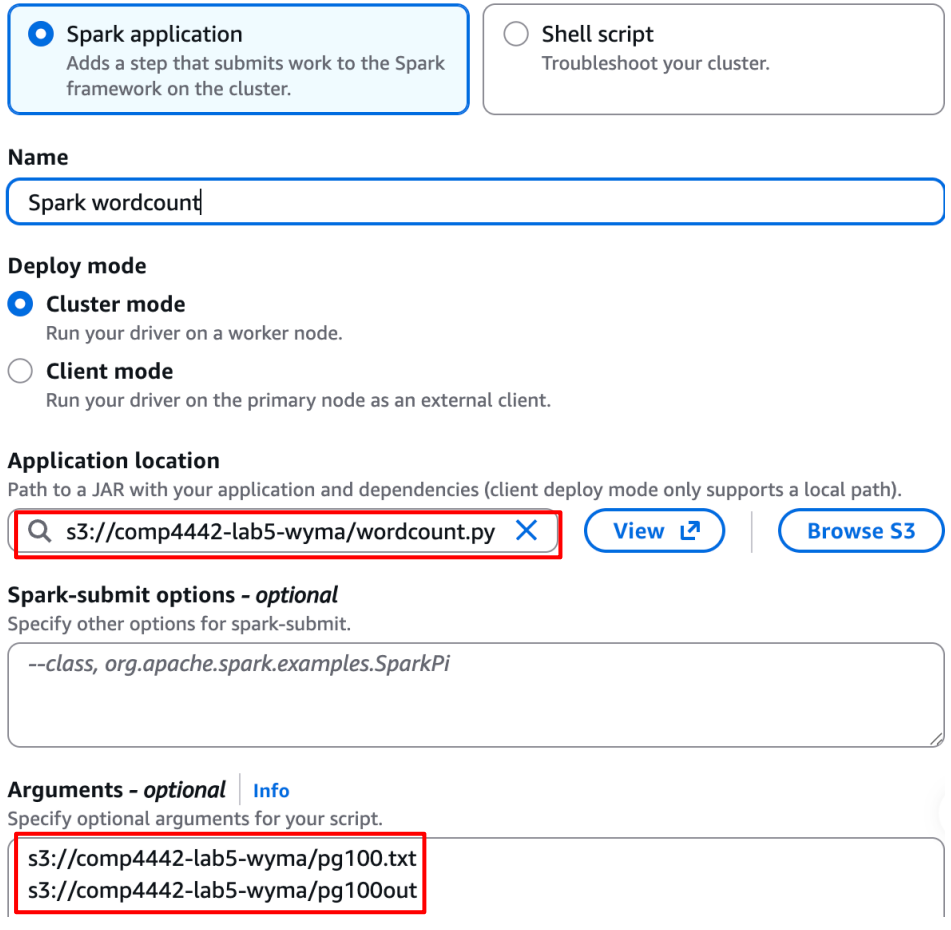
- How to interact with Spark with command line tools?

```
aws emr add-steps \
--cluster-id j-2AXXXXXXGAPLF \
--steps Type=CUSTOM_JAR,Name="Run spark-submit using command-runner.jar",ActionOnFailure=CONTINUE,Jar=command-runner.jar,Args=[spark-submit,S3://DOC-EXAMPLE-BUCKET/my-app.py,ArgName1,ArgValue1,ArgName2,ArgValue2]
```

https://docs.aws.amazon.com/emr/latest/ReleaseGuide/emr-commandrunner.html

Using command line tools

- How to interact with Spark with command line tools?




```
aws emr add-steps --cluster-id your_ID --steps Type=CUSTOM_JAR,Name="Run spark-submit using command-runner.jar",ActionOnFailure=CONTINUE,Jar=command-runner.jar,Args=[spark-submit, s3://your_bucket_name/wordcount.py, s3://your_bucket_name/pg100.txt, s3://your_bucket_name/pg100out] --region us-east-1
```

Using command line tools

- How to interact with Spark with command line tools?
- $ bash Remember to switch to bash in macOS, because zsh environment is not supported.
- $ aws emr add-steps --cluster-id j-1I59F4QR8BMEF --steps Type=CUSTOM_JAR,Name="Run spark-submit using command-runner.jar",ActionOnFailure=CONTINUE,Jar=command-runner.jar,Args=[spark-submit,s3://comp4442-lab5/wordcount.py,s3://comp4442-lab5/pg100.txt,s3://comp4442-lab5/pg100out] --region us-east-1

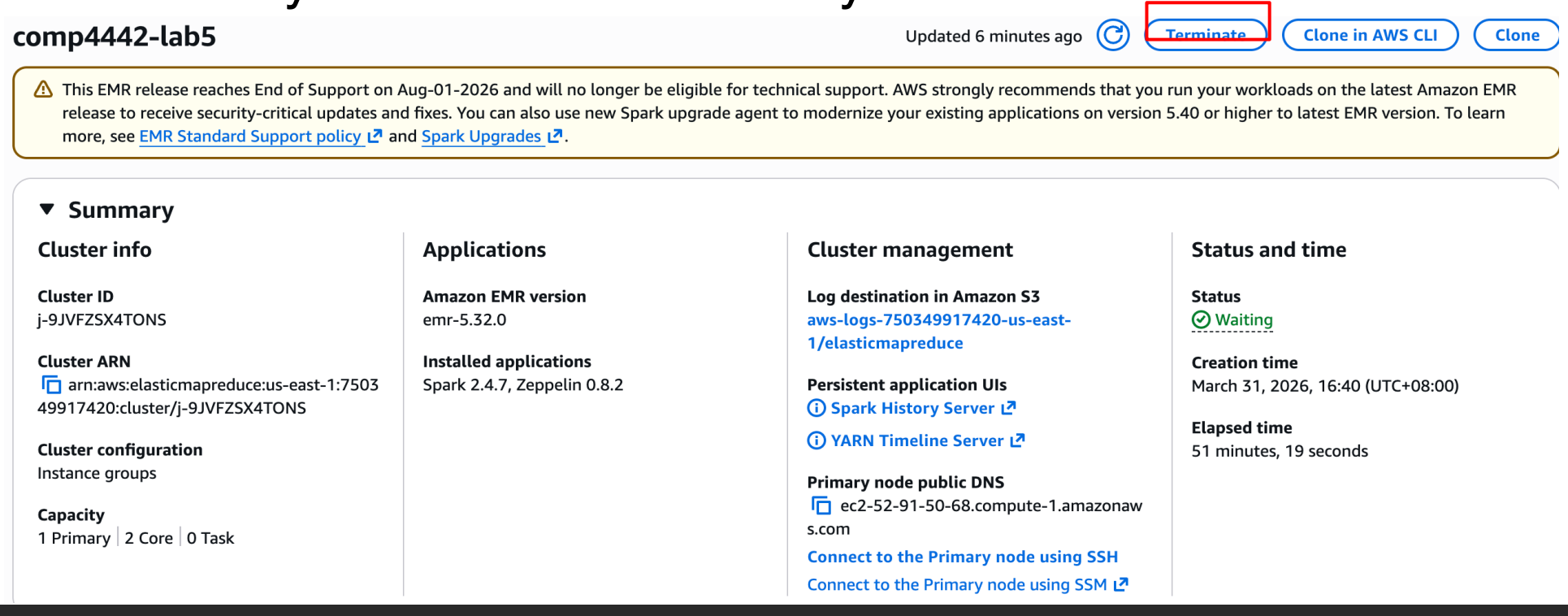
```
→ ~ % bash

The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
%F{green}→ %1~ %# %faws emr add-steps --cluster-id j-9JVFZSX4TONS --steps Type=CUSTOM_JAR,Name="Run spa
rk-submit using command-runner.jar",ActionOnFailure=CONTINUE,Jar=command-runner.jar,Args=[spark-submit,
s3://comp4442-lab5-wyma/wordcount.py,s3://comp4442-lab5-wyma/pg100.txt,s3://comp4442-lab5-wyma/pg100out
] --region us-east-1
```

Step ID	Status	Name	Log files
s-0101620308HG13C8XTEW	Running	Run spark-submit using command-runner.jar	No logs created yet
s-07619471NC255LUHX6ZJ	Completed	Spark wordcount	controller | syslog | stderr | stdout

```
{
    "StepIds": [
        "s-0101620308HG13C8XTEW"
    ]
}
```

Stop Spark Cluster

- Do not forget to terminate the cluster
  - The AWS will save the meta-data of the cluster for about two month
  - You can easily recover the cluster by clone it



Q&A

Exercise

- What is Narrow Dependencies and Wide Dependencies of RDDs? Which RDDs operations belongs to Narrow Dependencies and Wide Dependencies, respectively?

- What is lazy evaluation of RDDs? What is the benefits of lazy evaluation?